Digital Storemesh Co., Ltd. (Head Office)
131 Innovation Group Building 1, Room No. INC1-217, Moo 9, Khlong Nueng, Khlong Luang, Pathum Thani 12120

Full-Stack Developer Technical Assignment

Position Full-Stack Developer

Direction

1. Complete the Full-Stack developer technical assignment (as outlined in the attached document)
2. Please submit the related deliverables to company email at contact@storemesh.com within three days from the day you receive this assignment.

Digital Storemesh Co., Ltd. (Head Office)
131 Innovation Group Building 1, Room No. INC1-217, Moo 9, Khlong Nueng, Khlong Luang, Pathum Thani 12120

Full-Stack Developer Technical Assignment

Project Overview

The objective of this assignment is to develop a **StoreFront Management System**, a web application that facilitates the interaction between sellers and buyers. This project is designed to evaluate your proficiency in architectural design, database modeling, and full-stack implementation using modern frameworks.

Technical Specifications

Component	Technology	Requirement
Backend	Python	Django (Django REST Framework preferred)
Frontend	JavaScript/TypeScript	React.js with TypeScript
Database	Relational	Candidate's choice (e.g., PostgreSQL, SQLite)
Testing	Framework-native	Unit tests for at least Backend

User Stories

- **Authentication & Authorization:** As a user, I can register and log in to the system, choosing between two roles: **Seller** or **Buyer**. My access to features should be restricted based on this role.
- **Product Listing (Seller):** As a Seller, I can create, edit, and delete product listings. Each listing must include an image (file upload), title, description, unit price, and available quantity.
- **Marketplace Browsing (Buyer):** As a Buyer, I can view a list of all available products, apply filters to narrow the list, and click on a product to view its specific details (description, price, and stock status).
- **Inventory Management:** As a Buyer, when I "purchase" an item (a simple mock action), the quantity available for that product should decrease accordingly in the system.
- **Order Placement (Buyer):** As a Buyer, I can add one or more items to a shopping cart and proceed to a checkout process to finalize my order.

Project Deliverables

1. Database Design

Submit an **Entity-Relationship (ER) Diagram** representing the database schema you designed. This should clearly illustrate tables, relationships (one-to-many, many-to-many), primary keys, and foreign keys.

2. Implementation

- **Source Code:** A clean, modular codebase hosted on a Git repository.
- **Unit Tests:** comprehensive test suites ensuring core business logic (e.g., role-based access, inventory updates) remains stable.

- **Documentation:** A `README.md` file containing setup instructions, environment variable requirements, and a brief explanation of your architectural choices.

3. CI/CD Pipeline (Optional)

Implement a basic CI/CD pipeline (e.g., GitHub Actions) to automate the execution of tests upon code submission. This is not mandatory but highly regarded.

Evaluation Criteria

- **Architecture:** Effective separation of concerns between the frontend and the backend API.
- **Code Quality:** Adherence to Clean Code principles, naming conventions, and proper TypeScript typing.
- **Database Integrity:** Logic and efficiency of the ER diagram and schema implementation.
- **Testing Coverage:** Meaningful tests that cover critical paths rather than just high-level components.
- **Git Practices:** Meaningful and clear commit history, an effective branching strategy, and a properly configured `.gitignore` file.
- **Security:** Implementation of robust authentication (e.g., JWT for API access) and adherence to general security best practices.
- **Robustness and Error Handling:** Clear implementation of input validation and appropriate HTTP status codes for API responses.